ZAKPY ACADEMY
Votre réussite, notre mission.

PROJET ARDUINO

Thermomètre numérique avec alarme réglable

Un projet Arduino Mega avec un capteur LM35, un afficheur 7 segments et deux timers

NIVEAU	CARTE	CODE SOURCE
Intermédiaire	**Arduino Mega 2560**	**GitHub**
bases Arduino conseillées	ou simulation Wokwi / Proteus	zakariarhiba/ARDUINO-PROJECTS

1 L'idée du projet

Dans ce projet on va construire un petit thermomètre numérique avec un Arduino Mega. Il mesure la température avec un capteur LM35 et il l'affiche au dixième de degré sur un afficheur 7 segments, par exemple 24.8.

On ajoute aussi une alarme : avec un potentiomètre on choisit une température limite entre 0 et 40 °C. Si la température dépasse cette limite, une LED se met à clignoter.

Le projet a l'air simple, mais il permet de voir des notions qu'on utilise partout en électronique embarquée : la conversion analogique numérique, le multiplexage d'un afficheur, les registres du microcontrôleur, les timers et les interruptions.

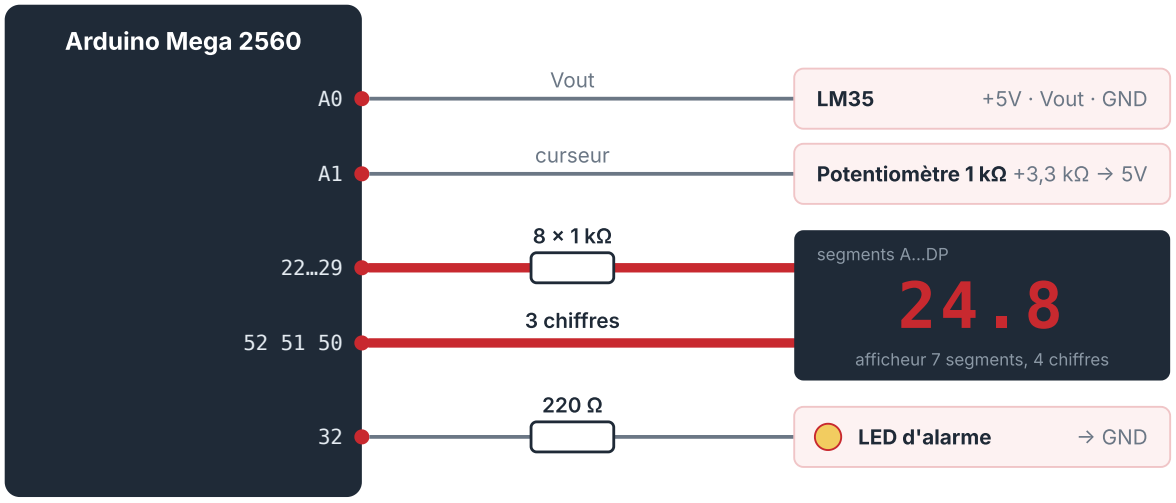
2 Ce que le thermomètre doit faire

- Mesurer la température avec une précision de 0,1 °C.
- Afficher la valeur sur 3 chiffres (format xx.x) et la rafraîchir en continu.
- Envoyer les mesures sur le moniteur série pour les vérifier.
- Régler une température d'alarme entre 0 et 40 °C avec un potentiomètre.
- Faire clignoter une LED quand la température dépasse l'alarme.

3 Le matériel

COMPOSANT	QUANTITÉ	REMARQUE
Arduino Mega 2560	1	avec son câble USB
Capteur de température LM35	1	sortie 10 mV par °C
Afficheur 7 segments, 4 chiffres	1	12 broches, par exemple 5641AS (cathode commune)
Potentiomètre 1 kΩ	1	pour régler l'alarme
LED	1	la couleur qu'on veut
Résistances 1 kΩ	8	une par segment
Résistance 3,3 kΩ	1	en série avec le potentiomètre
Résistance 220 Ω	1	pour la LED
Breadboard et fils		

4 Le câblage



ÉLÉMENT	BROCHE ARDUINO	REMARQUE
LM35 : +Vs, Vout, GND	`5V, A0, GND`	face plate vers soi : +5 V à gauche, Vout au milieu, GND à droite
Potentiomètre (curseur)	`A1`	le haut passe par la résistance de 3,3 kΩ vers le 5 V
Segments A, B, C, D, E, F, G, DP	`22 à 29`	chacun à travers une résistance de 1 kΩ
Chiffres dizaines, unités, dixièmes	`52, 51, 50`	le 4e chiffre (le plus à gauche) n'est pas utilisé
LED	`32`	à travers la résistance de 220 Ω vers GND

BROCHE DE L'AFFICHEUR	11	7	4	2	1	10	5	3	9	8	6
Rôle	A	B	C	D	E	F	G	DP	Ch.2	Ch.3	Ch.4
Arduino	22	23	24	25	26	27	28	29	52	51	50

ATTENTION

Si le LM35 chauffe dès qu'on branche, il est monté à l'envers : on débranche tout de suite. Et on ne met pas de résistance sur les broches des chiffres, seulement sur les segments.

POURQUOI LES BROCHES 22 À 29

Sur l'Arduino Mega, ces 8 broches forment le PORTA du microcontrôleur, dans l'ordre. Ça permet d'allumer les 8 segments d'un chiffre en une seule ligne de code : `PORTA = 0x5B;` affiche un 2.

5 Les étapes du projet

On construit le projet petit à petit. À chaque étape on teste avant de passer à la suivante. Le code de chaque étape est dans le dossier `steps` du projet sur GitHub.

Étape 1 Lire la température

On lit le LM35 sur l'entrée A0 et on affiche la température sur le moniteur série. On utilise la référence interne de 1,1 V de l'ADC : un pas fait alors 1,07 mV, ce qui donne environ 0,1 °C. Le calcul `N × 1100 / 1024` donne directement la température en dixièmes de degré. On en profite pour faire un test des segments qui affiche 88.8 au démarrage.

Résultat : « Temp = 24.8 C » toutes les 2 secondes, et l'afficheur clignote 3 fois avec 88.8.

Étape 2 Afficher sur le 7 segments avec le Timer 1

Les segments sont partagés entre les chiffres, donc on ne peut allumer qu'un chiffre à la fois. On les allume chacun leur tour, 5 ms chacun, assez vite pour que l'œil voie les 3 en même temps : c'est le multiplexage. Pour que ça tourne tout seul, on le confie au Timer 1 qui déclenche une interruption toutes les 5 ms (prescaler 64, OCR1A = 1249).

Résultat : la température s'affiche en continu et suit le capteur.

Étape 3 Passer sur la vraie carte

On câble le capteur et l'afficheur sur la breadboard et on charge le même programme. La seule chose à vérifier, c'est le type d'afficheur : avec une cathode commune le code marche tel quel, avec une anode commune il faut inverser les niveaux des segments et des chiffres.

Résultat : le thermomètre fonctionne en vrai. On pince le capteur avec les doigts et la température monte.

Étape 4 Régler l'alarme avec le potentiomètre

On lit le potentiomètre sur A1 et on transforme la valeur en une température entre 0 et 40,0 °C avec `map()`. Ensuite on compare : si la température dépasse l'alarme, la LED s'allume. Le moniteur série affiche les deux valeurs.

Résultat : on tourne le potentiomètre sous la température et la LED s'allume.

Étape 5 Faire clignoter l'alarme avec le Timer 3

Au lieu d'une LED fixe, on veut qu'elle clignote. On utilise un deuxième timer, le Timer 3, réglé sur 0,5 s (prescaler 256, OCR3A = 31249). Sa fonction d'interruption change l'état de la LED quand l'alarme est active. La boucle `loop()` se contente de mesurer et de décider.

Résultat : le thermomètre complet. La LED clignote une fois par seconde quand la température dépasse l'alarme.

6 Les points techniques du projet

La précision de 0,1 °C

RÉFÉRENCE DE L'ADC	1 PAS	EN TEMPÉRATURE
5 V (par défaut)	4,88 mV	0,49 °C
1,1 V (INTERNAL1V1)	**1,07 mV**	**0,107 °C**

Le codage des chiffres

Chaque segment correspond à un bit : du bit 0 (A) au bit 7 (DP). Les codes de 0 à 9 sont `0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F`, et on ajoute le point avec `| 0x80`.

Le calcul des timers

Un tick dure P / 16 µs, et la valeur cible se calcule avec **OCR = (16 000 000 / P) × temps − 1**, sans dépasser 65 535.

TIMER	RÔLE	PRESCALER	OCR	PÉRIODE
Timer 1	multiplexage de l'afficheur	64	1249	5 ms
Timer 3	clignotement de la LED	256	31249	0,5 s

7 Tester sans matériel

Pas encore de carte ? Le projet marche aussi sur **Wokwi**, un simulateur gratuit dans le navigateur. Le dossier `wokwi` du projet contient le circuit (`diagram.json`) et une version du code adaptée : comme Wokwi n'a pas de LM35, un potentiomètre sur A0 joue le rôle de la température.

8 Pour aller plus loin

- Remplacer la LED par un buzzer pour avoir une vraie alarme sonore.
- Ajouter une hystérésis pour que l'alarme ne s'allume et s'éteigne pas sans arrêt autour de la limite.
- Garder la limite en mémoire (EEPROM) même après une coupure de courant.
- Ajouter un bouton pour passer en degrés Fahrenheit.
- Envoyer la température sur le téléphone, une première brique pour le projet de maison connectée King House.

RHIBA Zakaria
Zakpy Academy. Le code complet est sur GitHub : github.com/zakariarhiba/ARDUINO-PROJECTS